

Analysis: Append Sort

Test Set 1

First of all, note that it is never optimal to append anything to the first number. Now, with the low limits we can iterate through all the ways to append digits to the second and third numbers while achieving the problem goal. We should stop once we understand that we cannot do better, for example, when both numbers become longer than 4 digits.

Test Set 2

We can use a greedy approach to solve the problem: for every number starting with the second one, make it larger than the previous one but as small as possible. Note that this ensures we use as few digits as possible for the current number and at the same time makes it as easy as possible for the next number. This means the greedy choice is optimal.

So the problem now is how do we compute this efficiently. More formally: given integers A and B , find the minimum B' such that $B' > A$ and B is a prefix of B' . Let us denote the number of digits in an integer Z as $\text{len}(Z)$.

First of all, if $A < B$, we can set $B' = B$.

Now let's consider the case when $A \geq B$ and $\text{len}(A) = \text{len}(B)$. Note that appending any digit to B will make it larger than A . To make it as small as possible we can just append a zero.

The only case that is not considered now is when B has fewer digits than A . Let $k = \text{len}(A) - \text{len}(B)$.

First, we can try making B' equal to $B \times 10^k$, that is, append k zeroes to the right of B . If such a B' is larger than A , then this is the optimal solution.

Now, we check if it is possible that B' is the same length as A . If appending k nines to B doesn't result in something larger than A , then B' will need to have more digits than A . In such a case, we can just make B exactly 1 digit longer A and as small as possible by appending $k + 1$ zeroes to B .

If appending k zeroes to B is too small but appending k nines makes it larger than A , then B is actually a prefix of A . In such a case $B' = A + 1$ is the optimal answer.

All the checks performed above are linear in the length of A and B and each number in the input is processed at most once as A and at most once as B . However, numbers may become longer after each operation, but only by 1 digit. Therefore, the complexity of the overall algorithm is quadratic in the total number of digits in the input, and linear in the number of digits of the output. As an example, an input of N strictly decreasing integers of the same length yields an output where the i -th integer consists of exactly one more digit than the previous, which needs $(N \cdot (N - 1))/2$ operations in total.

Analysis: Hacked Exam

In this problem we want to maximize our expected score. Let p_q be the probability of question q 's answer being $\mathbb{T}$, and let $Q_{\mathbb{T}}$ and $Q_{\mathbb{F}}$ be the sets of questions we answer with $\mathbb{T}$ and $\mathbb{F}$, respectively. By [linearity of expectation](#), the expected score to maximize can be written as

$$\left(\sum_{q \in Q_{\mathbb{T}}} p_q \right) + \sum_{q \in Q_{\mathbb{F}}} (1 - p_q).$$

Notice that linearity of expectation can be used regardless of any conditional probability between different questions. In this case, those conditional probabilities can be quite complicated, so being able to ignore them and treat each question independently despite them not being [independent in the probability theory sense](#) simplifies the solution process significantly. This is a trick that is important in many problems about expected values.

We operate with fractions throughout most of the proposed solutions. As noted in the sample, the needed numbers may exceed 64 bits for Test Set 3, and potentially for Test Set 2 as well, depending on the exact implementation of the algorithm and the fraction operations. It is absolutely possible to solve Test Set 2 with simply 64-bit integers and Test Set 3 with just 128-bit integers. Our most popular languages all support this in some way: C and C++ have `__int128` support, Java and C# have `BigInteger`, JavaScript has `BigInt`, Bash has `bc`, and Python, Ruby and Haskell have native long integer support. We usually strive to make most of our problems solvable with just 64-bit integer arithmetic, but in this case, limiting the number of questions so much would allow suboptimal solutions in fast languages to pass Test Set 3.

Notice that we can have information of 1 or 2 other students in the first two test sets, and also 3 in the last test set. We could solve each number of students independently, but there is no need for that. Adding a student with the same answers and score as a student in the input results in a completely equivalent case, so we can always assume that there are a maximum number of students by copying any student the needed number of times.

Test Set 1

In Test Set 1, the number of questions Q is small enough that we can simply enumerate all possible 2^Q sequences of answers, and filter out those who are inconsistent with the input (i.e., those for which one of the students would obtain a different score than they actually got). From the consistent ones, we can estimate the p_q s above as the ratio between the number of sequences that answer $\mathbb{T}$ to question q , over the total. Then, we can simply choose to answer $\mathbb{T}$ to those questions with $p_q > \frac{1}{2}$ and $\mathbb{F}$ to those with $p_q < \frac{1}{2}$. We can answer the questions with $p_q = \frac{1}{2}$ either way.

Test Set 2

In Test Set 2 Q is large, so we cannot enumerate the sequences of answers. We can, on the other hand, figure out the probabilities p_q in a different way, and then proceed as before with choosing the answers by comparing those values to $\frac{1}{2}$.

An insight-based solution

One way to solve Test Set 2 is by splitting the questions into types. If two questions q_1 and q_2 received the same answer from each student, then by symmetry $p_{q_1} = p_{q_2}$. Then, let p_{ab} be equal to the probability of a question's answer being $\mathbb{T}$ given that the first student answered a and the second student answered b to it. By the first observation, each p_q is equal to one of the 4 values $p_{\mathbb{T}\mathbb{T}}$, $p_{\mathbb{T}\mathbb{F}}$, $p_{\mathbb{F}\mathbb{T}}$ or $p_{\mathbb{F}\mathbb{F}}$. Moreover, by the symmetry of complementing answers, $p_{\mathbb{T}\mathbb{T}} = 1 - p_{\mathbb{F}\mathbb{F}}$ and $p_{\mathbb{T}\mathbb{F}} = 1 - p_{\mathbb{F}\mathbb{T}}$. Therefore, we can express every p_q as a linear function of up to two variables $p_{\mathbb{T}\mathbb{T}}$ and $p_{\mathbb{T}\mathbb{F}}$. That means we can express the expected score of both students as linear functions on those two variables too. Given that their expected score should match their actual score, that gives us two equations with two unknowns. We can derive the real values of $p_{\mathbb{T}\mathbb{T}}$ and $p_{\mathbb{T}\mathbb{F}}$ from that system of equations. With every p_q calculated, we can simply choose, for each question, an answer that has maximum probability, as in the solution for Test Set 1.

Notice that there are 4 possible cases depending on how our two variables compare with $\frac{1}{2}$ (if one is exactly $\frac{1}{2}$ that means two cases are equivalent and if both are $\frac{1}{2}$ then all cases are equivalent). The 4 cases exactly match with either answering the same as one of the students, or answering the opposite of one of the students. We can use this observation to greatly simplify the implementation as the score of a sequence given by a student is given to us, and the score of the complement of the answers of student i is $\mathbf{Q} - \mathbf{S}_i$, so we can easily obtain the scores of the 4 options and pick a highest one.

A more competitive-programming-standard solution

In case of having 2 students, we can use [dynamic programming](#) to calculate the probability of each question having a particular answer. We compute the recursive function $f(s_1, s_2, q)$ defined as "how many ways are there to answer questions $q, q+1, q+2, \dots, \mathbf{Q}$ such that student 1 gets exactly s_1 of them right and student 2 gets exactly s_2 of them right?" We can define that function recursively as

$$f(s_1, s_2, q) = f(s_1 - I_1(q, \mathbb{T}), s_2 - I_2(q, \mathbb{T}), q+1) + f(s_1 - I_1(q, \mathbb{F}), s_2 - I_2(q, \mathbb{F}), q+1)$$

where $I_i(q, c)$ is 1 if student i answered c to question q and 0 otherwise. The base cases are $f(0, 0, \mathbf{Q}+1) = 1$ and $f(s_1, s_2, \mathbf{Q}+1) = 0$ whenever one of s_1 or s_2 is not 0. To simplify memoization, we may want to add a case to simply answer 0 whenever s_1 or s_2 are negative, but the recursive equation holds as written for those cases.

By memoizing that function, we can compute it in time $O(\mathbf{Q}^3)$. We can calculate the probability of question 1's answer being $\mathbb{T}$ as

$$\frac{f(\mathbf{S}_1 - I_1(1, \mathbb{T}), \mathbf{S}_2 - I_2(1, \mathbb{T}), 2)}{f(\mathbf{S}_1, \mathbf{S}_2, 1)}.$$

Then, by symmetry, we can reorder the questions to make any question the first one and re-run to compute the probability for any question. After having all probabilities, we simply answer the most likely answer for each question and sum its probability to our expected score. Since we need to run the probability computation $O(\mathbf{Q})$ times (once per question), the overall algorithm takes $O(\mathbf{Q}^4)$ time. This can be a little too slow.

If we add only the first observation of the insight-based solution, we can notice that two questions that were answered the same by both students have identical probabilities. Then, we only need to calculate the probability of up to 4 question types (in the notation of the previous solution, we use dynamic programming to calculate all the p_{ab} s). This improves the overall running time to $O(\mathbf{Q}^3)$, which fits better within the time limit. An observation about complement

could further reduce this to only 2 question types, but that does not change the time complexity and it is not needed to pass this test set.

Test Set 3

The dynamic programming solution for Test Set 2 can be generalized to Test Set 3 by adding an additional score as another parameter to the recursive function. However, the additional dimension and larger limit for Q can easily make such solutions too slow.

Combining the full insights of the first solution to Test Set 2 with the solution to Test Set 1 works, though: there are 8 probability variables p_{abc} , and pairs of complementary variables have complementary probabilities, so we only care about 4 different ones.

Let us call the subindex of the variables (the abc part) the "type" of a question. Let us number the types 1 through 4 in any order. If there are q_j questions of type j , we can use quadruples (t_1, t_2, t_3, t_4) with $0 \leq t_j \leq q_j$ for all j to represent sequences of answers that answer $\mathbb{T}$ to exactly t_j questions of type j . We know that there are

$$\binom{q_1}{t_1} \cdot \binom{q_2}{t_2} \cdot \binom{q_3}{t_3} \cdot \binom{q_4}{t_4}$$

sequences of answers represented by this particular quadruple. If we filter the quadruples by the ones that give each student their actual score, we are effectively enumerating answers that are consistent with the input. This is what we did for Test Set 1! In this way, we can count which amount t_j of questions of type j has the largest probability and choose that one, for each j .

There are at most $(Q/4)^4$ quadruples to check. This makes the time complexity of the algorithm $O(Q^4)$, but the $1/256$ constant is pretty significant, and a good implementation runs comfortably in time.

But wait! We can refine this solution even more by using the solution for Test Set 2 that ends with solving the system of equations! We can express the score of each student as a linear function of t_1, t_2, t_3 and t_4 . That gives us a system of 3 equations and 4 unknowns. That means that we only need to try all possible values for one of the t_j and then simply solve the system to find unique values for the other three. That refines the solution above to requiring only $O(Q)$ time.

Analysis: Prime Time

Test Set 1

In Test Set 1, the total number of cards in the deck is at most 10. A small number like 10 suggests we can brute force and simulate all possible game scenarios. For each card we can decide whether it belongs to the first group or the second group. Next, we sum up the numbers on the first group cards and compare with the product of the numbers on the second group cards. If they are equal, current sum or product is one of the possible candidate scores. After checking all possible partitions, we output the maximum score or output the score 0 if there is no feasible way to partition the cards.

Two things to watch out for. First, each of the groups has to be non-empty. This is easy to handle. Second, the product of all the numbers may overflow the integer type of our choice (in the worst case there may be 10 cards, each with 499). There are many ways one can avoid this problem. One way can be to first find the sum of the first group numbers and then try to divide it by the second group numbers, one by one. If at some point we are unable to perform the division or if after all the divisions the result is not 1, we know current partition candidate can not be good. Another way would be to notice that the sum of the first group can not be very high. A naive upper bound is 4990 (all the 10 numbers are 499 and they are in the first group). Hence we can find the product of the second group numbers in floating point and compare with the sum of the first group numbers. Since the upper bound of the sum of the first group numbers is very low, precision error should not cause any trouble.

There are 2^N ways to partition the cards where N is the number of cards. After each partition we can find the sum and the product of the groups with a linear loop. Hence the total run time is $O(2^N N)$. We could also get rid of the linear loop but such optimization was not necessary.

Test Set 2

In Test Set 2, the total number of cards is at most 100. Performing all possible partitions would surely timeout. But notice that we cannot put many numbers into the second group because the product of positive numbers greater than 1 quickly increases. This time the upper limit for the sum of the first group numbers is 49900. That means, we can not have more than 15 numbers in the second group ($2^{16} > 49900$). We can use this observation to optimize our brute force solution.

Let S be the set of all possible partitions using the first i cards for some arbitrary ordering of the cards. Instead of keeping explicit partitions, we will represent the partition as (x, y) where x is the sum of the numbers in the first group and y is the product of the numbers in the second group. This helps us to reduce the state space. Once we partition the first i cards, it does not matter exactly which card went to which group. We just care about the sum and the product of the respective groups. Now let b be the value of the $i + 1$ 'th card. Hence for each partition (x, y) in S we will have two choices: $(x + b, y)$ and $(x, y \times b)$. If this makes the product of the second group numbers larger than 49900, we prune this state. After considering all N cards, we check if there is some partition where the sum of the first group numbers is equal to the product of the second group numbers (i.e., a tuple in S where both values are equal). We output the maximum of these sum or product, or 0 if there is no such partition.

We can have at most 100 numbers in the input. So it might be tempting to think that there may be $\binom{100}{15}$ possible valid partitions. But the product of the numbers in the second group can be at

most 49900 and each of these 49900 numbers can be uniquely represented as the product of primes. So the remaining numbers will go to the first group. Hence after considering each of the input numbers there can be at most 49900 possible partitions (49900 possible second groups and for each of these there is a unique first group). So this naive calculation gives us a limit of $49900 \times 100 \approx 5 \times 10^6$ operations.

So far we have used the fact that each score can be uniquely represented as a product to prove that our solution is fast enough, but it can also be used to obtain an alternative approach for Test Set 2: we have only 49900 candidates for the final score, so we can iterate over them one by one. For each candidate score, there is exactly one way to represent it as a product of primes. Assuming we know the prime factorization of the candidate how can we determine if the candidate is valid? Suppose the prime p appears in the prime factorization q times. Then, the prime p has to be in the input at least q times. This condition guarantees us that the second group is achievable. Next we need to come up with a condition to check the existence of the first group. If we know the sum of the input numbers and subtract the sum of the numbers in the second group we can get the sum of the first group which has to be equal to our candidate. This guarantees the first group existence. Both of these conditions can be checked very easily given the prime factorization of the candidate score.

It turns out that this approach generalizes very well to Test Set 3.

Test Set 3

In Test Set 3, N can be up to 10^{15} , which means the sum of the first group can be as high as 4.99×10^{17} . But the number of cards in the second group can not be very high. Since $2^{60} > 4.99 \times 10^{17}$, we can consider 60 as the upper bound of number of cards in the second group. Although the product of at most 60 cards in the second group can range from 2 to 2^{60} , the sum of the second group numbers can be only up to $60 \times 499 = 29940$ (the actual maximum possible sum of the second group numbers is 3025 under the problem's constraint but even a crude estimate of 29940 is enough to get a working solution). Let X be the sum of all the cards in the input. Then, the sum of the first group numbers must be between $X - 29940$ and X , inclusive. This means we have only 29941 candidates for the final score, so we could apply the second approach from Test Set 2 if only we could factorize those candidates.

Unfortunately factorizing a number as big as 10^{17} is not an easy task. A naive way may be to run a loop up to $\sqrt{10^{17}}$. However, in this problem we do not need to care about the primes higher than 499. To put it another way, if the candidate has a prime factor other than the input prime numbers, we cannot achieve this score in the second group. Hence, it is enough to try to factorize the 29940 candidates with only primes from 2 to 499. It takes about $29940 \times (95 + 60) \approx 4.6 \times 10^6$ operations to factorize the 29940 candidates (there are 95 primes between 2 and 499 and in total there can be at most 60 prime factors). We could also do [sieve](#)-like factorization reducing the number of operations to about $29940 \log \log 499 \approx 10^5$ but this is not necessary. After the factorization, we can run a loop to check if the number of each prime exceeds the input count and also to sum these primes. This takes $29940 \times 95 \approx 3 \times 10^6$ operations.